

Week-4 (DL Gen AI)

L1 Famous CNN Architectures

- ImageNet Large Scale Visual Recog. Challenge (ILSVRC) annual comp. → task of classifying an image in 1000 categories.

- Metric → top-5 error rate.

→ it lead SOTA CNN architectures.

- AlexNet → won in 2012

huge jump

error rate → 16% ← 28%

$227 \times 227 \times 3$

conv 1 ($55 \times 55 \times 96$)

↓

conv 2 ($27 \times 27 \times 96$)

↓

conv 3 ($27 \times 27 \times 256$)

↓

conv 4 ($13 \times 13 \times 256$)

⋮

conv 8

↓

FC layers

- total

60M

parameters

btw.
we have
max
pool
in btw.

→ used 2 GPUs

→ use of ReLU activation & dropouts

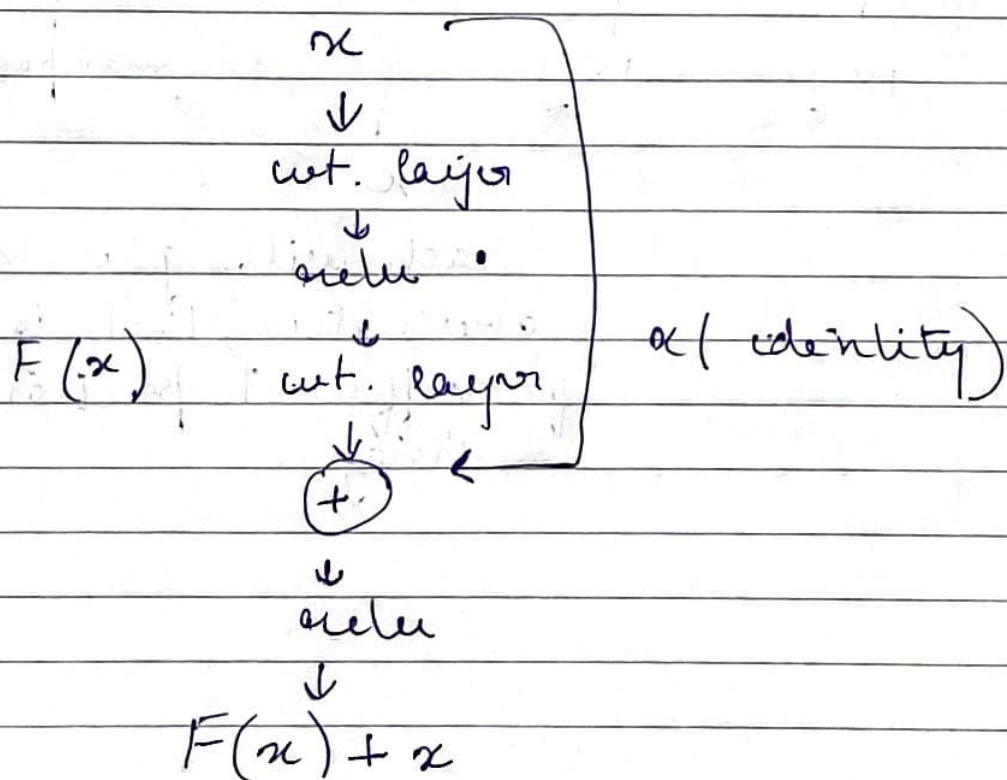
- VGGNet → proved that extreme network depth (16-19 layers) was imp. for performance, setting a new std.

//_

→ exclusive usage of 3×3 kernels → when
since stacked up, gives a larger effective
receptive field.

- ResNet → Residual blocks, features a skip
connecⁿ. Adds block's input directly
to its output $F(x)$.
$$F(x) = H(x) - x \Rightarrow \text{residual funcⁿ}$$

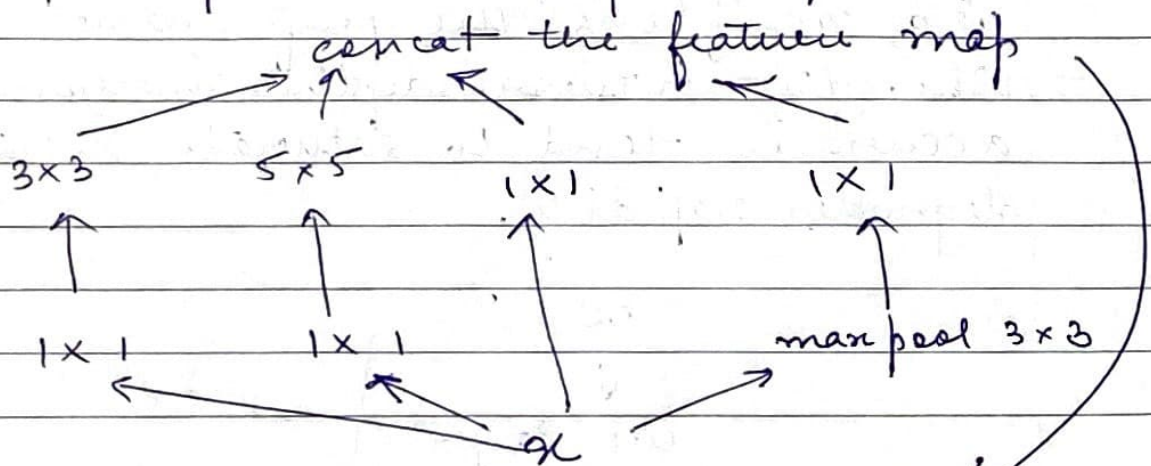
→ helps to learn identity funcⁿ by deriving
the wts. of $F(x)$ to 0. Solved the problem
of degradatⁿ problem & allowed the training
of extremely deep networks (152 for ResNet)
w/o sacrificing the performance.
- When the network depth increases,
accuracies tend to saturate & then later
degrades rapidly.



- _/_/_
- GoogleNet $\rightarrow$ "inception" module $\rightarrow$ performs parallel convoluⁿ (1×1 , 3×3 , 5×5) & pooling on the same input & finally concatenate their feature maps. Uses 1×1 bottleneck layer to perform dimensionality reducⁿ before the expensive 3×3 or 5×5 convolutions.

$\rightarrow$ It is computationally efficient with much less parameter count.

$\rightarrow$ It also replace the FC layers with a single GAP (Global Avg. Pooling) w/o having to reducing the parameters.



each will give different attributes, that is inpt. for different positions of image.

L2 Transfer Learning

- how to utilize arch. for specific tasks.
- Training a large CNN requires huge datasets & also, computational resources.
- TL → take pre-trained model & fine-tuning it for a new specific task.
 - use pretrained convolutional base.
 - remove the original classifier head.
 - add your own small classifier head
 - train only your own small head.
- it is std. & most used practice these days in vision model training.
- You can use VGGNet, etc and use it as a feature extractor head. Freeze the nets & add a new classifier head. This strategy works if your new dataset is small.
- We initialize the model with pre-trained nets. Unfreeze & use the new data to retrain partially or completely ⇒ Fine-tune
- Pre-trained backbone → start with network like ResNet or MobileNet.
 - remove the top layer, initialize a new layer with std. no. of classes.
- For Segmentation, TL is used to build the encoder (UNet or FCN) which downsamples & a decoder part (which upsamples

- _/_/_
- use the pre-trained model like ResNet that replaces encoder path.
 - train the decoder head is added.

L3 Img. Classificaⁿ with TL: Introducⁿ

- we generally have lack of labelled data.
- TL helps in adaptⁿ of knowledge from 1 domain to another.

$$S = \{x_s, y_s\} \quad T = \{x_t, y_t\}$$

$f_\theta \rightarrow \text{NN with } \theta, \phi_x = \text{feature representaⁿ funcⁿ}$

$$L_{\text{transfo}} = L_{\text{target}}(f_\theta(x_t), y_t) + \lambda * R(\phi)$$

↙ balance parameter
 ↘ regularization term based on domain knowledge

- we can use this in multiple domains.
- we will use Food Vision Mini dataset. small dataset, 3 classes, imgs. resoluⁿ are super.
- we will use pre-trained models from torchvision models.

L4 Img. Classification with TL: Dataset Accumulation

- prepare the dataset
 - import the libraries including torch vision, data manipulation & viz libraries
 - download the data
 - source
 - destination
 - remove_source
 - returns the path object
 - great part, is this code works cross-platform
 - get the pizza-steak-sushi dataset

L5 Img. Classification with TL: Data Exploration

- comprehensive data statistics, sample counts, & dataset partitioning
- we keep the splits as → train & test
- count the main image files (using glob)
- perform this class balance analysis
 - check for each class & calculate the ratio
- visualize the data once for more clarity.

L6. Image Classification with TL: Data Preprocessing

- we have seen plots after downloading the dataset.
- we have to now make our input images which ResNet can work on.

↳ Image Transform

- We are using ImageNet here → so use its RGB means for pre-training models.
- so give images a slight rotation so that model becomes robust to such changes.
- we will use torchvision datasets & its dataloader to wrap up the things, so that batch-wise training can be done efficiently.
- the name of the folder → is generally the class name → works when you use the image loader.
- we don't use shuffle in the evaluation mode.

L7. TL: PreTrained Model Usage

- here we are using EfficientNet-B0

it is present in torchvision model.

- use the wts. of this model.
- efficientnet $\rightarrow$ features $\rightarrow$ main feature extractor
 - $\rightarrow$ avg. pool
 - $\rightarrow$ Classifier $\rightarrow$ FC layer
- $\rightarrow$ It has around 5.2 Million model.
- so we will freeze the layers of the model and only change the classification head.
 - $\rightarrow$ once you change the params gets reduced drastically to around 4 millions.

L8. TL: Training & Evaluation

- we bring the data transforms from efficient net and bring it on the ImageNet
- we use optimizers $\rightarrow$ Adam
loss $\rightarrow$ BCE loss
- update the params and take the optimizers steps towards the convergence.
- the funcⁿ returns the test & train accuracies for each epoch. $\rightarrow$ takes the avg. metrics.
- evaluate the model by plotting the curves for both train & test $\rightarrow$ loss & accuracies.

L9. Image Segmentation: Intro. to UNet

- UNet used for image segmentation.
 - ↳ Conference → leaf dataset
 - ↳ used in biomedical image segmentation
 - ↳ lot of convolution layers (with a similar encoder-decoder architecture)
- it uses skip-connections betw. corresponding encoder & decoder layers → it retains the spatial information lost during downsampling.
- it can generate pixel-level classification and captures very fine boundaries.

L10. Img. Seg.: Data Accumulation

- we will use kagglehub to download the dataset → it brings dataset w/o any API or boilerplate code.
- train & masks → segmentation for corresponding images.

L11. Img. Seg.: Data Preprocessing

- import CV2 & PIL

- we are exploring leaf-disease-segmentaⁿ imgs.
- examine the dimension & data types of imgs. & its corresponding segmented masks.
- visualize the dataset to understand how the disease is marked on the leaf.
- goal is to predict the mask for the given input image.
- Albumentations → advanced img. augmentaⁿ library that supports PyTorch integraⁿ.
- define the class for train & test.
→ partition the datasets

L12 Img. Seg. : Training & Evaluaⁿ

- custom datasegmentaⁿ class
 - dataloader → for data access
 - implement the data preprocessing pipeline
 - impt. convert mask to greyscale 'L'
 - define the transformations
- configure the data loader for model training.